Computer Security Symposium 2026
19-23 October 2026

コンテナ環境におけるBPF Tokenの ポリシー表現上の限界を補完する防御方式の提案と評価

木場 翔太[1,a)]　長谷川 皓一[2]　山口 由紀子[3]　嶋田 創[3]

概要：コンテナプロセスはホストカーネル上で実行されるため，その eBPF 利用には適切なセキュリティ制御が必要である．BPF Token は，コンテナプロセスに対し，許可する eBPF 操作を限定して権限を委譲する仕組みである．本研究では，BPF Token による権限委譲後の eBPF 操作に対する制御限界を実験的に検証した．その結果，BPF オブジェクト作成による資源消費，ヘルパー利用範囲，およびアタッチ対象とその実効範囲について，BPF Token 単独ではコンテナや Pod ごとのポリシーを表現できないことを明らかにした．そこで，コンテナプロセスに BPF Token を渡さず，ブローカーが要求を検査して eBPF 操作を代行する方式と，コンテナプロセスが Token を保持して直接行う eBPF 操作に追加のポリシーを適用する BPF LSM 方式を実装した．評価の結果，前者はより広範なポリシーを適用できるが処理コストが大きく，後者は適用範囲に制約があるが処理コストが小さいことを示した．

キーワード：eBPF，BPF Token，LSM

Addressing the Policy Limitations of BPF Token in Container Environments: Design and Evaluation of Complementary Defenses

Shota Kiba[1,a)]　Hirokazu Hasegawa[2]　Yukiko Yamaguchi[3]　Hajime Shimada[3]

Abstract: Container processes run on the host kernel; therefore, their use of eBPF requires appropriate security controls. BPF Token is a mechanism that can delegate permission to perform selected eBPF operations to container processes. In this research, we experimentally examined its post-delegation control limitations. We found that a standalone BPF Token cannot express per-container or per-Pod policies for resource consumption from BPF object creation, helper availability, or attachment targets and their effective scope. To alleviate this problem, we implemented two complementary approaches: a broker that retains the token, inspects requests, and performs eBPF operations on behalf of container processes; and a BPF LSM approach that enforces additional policies on direct eBPF operations by token-holding container processes. Experimental results show that the broker approach supports broader policies with higher overhead, whereas the BPF LSM approach provides narrower coverage with lower overhead.

Keywords: eBPF，BPF Token，LSM

[1] 名古屋大学大学院情報学研究科
Graduate School of Informatics, Nagoya University
[2] 国立情報学研究所ストラテジックサイバーレジリエンス研究開発センター
Center for Strategic Cyber Resilience Research and Development, National Institute of Informatics
[3] 名古屋大学情報基盤センター
Information Technology Center, Nagoya University
[a)] kiba@net.itc.nagoya-u.ac.jp

1. はじめに

eBPF（extended Berkley Packet Filter）は，カーネルのソースコードを変更することなく，カーネルの機能を動的に拡張する仕組みである．高速なパケット処理，システムおよびアプリケーションのトレーシングや性能解析，な

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

らびにプロセス実行やシステム資源へのアクセスに対するセキュリティ監視・ポリシー強制などに利用されており，クラウドネイティブ環境におけるネットワーク制御やコンテナ監視を支えている．

eBPF 操作に関する権限の制御は，各種の隔離の下で実行されるコンテナプロセスに対しても重要である．コンテナプロセスに許可する eBPF 操作の範囲が適切に制限されていない場合，コンテナ外の資源への作用や，コンテナ外のイベントの観測が可能となるおそれがある [1]．

BPF Token は，eBPF 操作に必要なケーパビリティをホスト全体に対して有効な形でコンテナプロセスに直接付与せず，eBPF 操作に関する権限を限定して委譲するために Linux 6.9 で導入された機構である [2]．BPF Token を用いることで，実行を許可する BPF コマンド，作成を許可する BPF マップの型，ロードを許可する eBPF プログラムの型，ならびに eBPF プログラムのロード時に指定するアタッチ型を制限できる．

一方，本研究では，BPF Token を用いて eBPF 操作に関する権限が正当に委譲された環境を対象とし，権限委譲後に残る制御限界に着目する．具体的には，コンテナや Pod ごとの制御への利用を想定し，資源消費，ヘルパー利用範囲，およびアタッチ対象とその実効範囲について，cgroup ID ごとのポリシー適用可否を検証する．さらに，コンテナプロセスに Token を渡さず，ブローカーがコンテナプロセスからの要求を検査して eBPF 操作を代行する方式と，コンテナプロセスが Token を保持して直接行う eBPF 操作に対し，BPF LSM（Linux Security Modules）で追加のポリシーを適用する方式を設計・実装する．両方式について，適用可能なポリシー，保護範囲，および処理コストを比較する．

本研究の主な貢献を以下に示す．

(1) BPF Token を用いて eBPF 操作に関する権限を正当に委譲する環境を対象とし，権限委譲後の制御限界を，資源消費，ヘルパー利用範囲，およびアタッチ対象とその実効範囲の 3 つの観点から明らかにした．
(2) 明らかにした制御限界を補完するため，eBPF 操作を代行するユーザ空間ブローカー方式と，直接的な eBPF 操作を制御する BPF LSM 方式を設計・実装した．
(3) 両方式について，適用可能なポリシー，保護範囲，および処理コストを評価し，トレードオフを示した．

2. 研究背景と関連研究

2.1 eBPF と BPF Token

eBPF は，カーネルのソースコードを変更することなく，カーネルの機能を動的に拡張する仕組みである．ユーザ空間のアプリケーションが eBPF プログラムをロードすると，カーネルの検証器は，制御フローやメモリアクセスなど，命令列を安全に実行するための条件を満たすか検査する．検査を通過したプログラムを各種のフックにアタッチすると，システムコールの呼出し，ネットワークパケットの処理，トレースポイントへの到達など，アタッチ先に対応する処理を契機としてカーネル空間で実行される [3]．

Linux では，特権を要する eBPF 操作の許可判定にケーパビリティが用いられる．かつては，このような操作の多くに，より広範な `CAP_SYS_ADMIN` が用いられていた．Linux 5.8 では，eBPF 操作および性能監視に関する権限を `CAP_SYS_ADMIN` から分離するため，`CAP_BPF` および `CAP_PERFMON` が導入された [4]．現在は，特権を要する eBPF 操作に `CAP_BPF` が要求される．プログラムのロードについては，`CAP_BPF` に加えて，XDP など一部のネットワーク関連プログラム型には `CAP_NET_ADMIN` が，kprobe や tracepoint など一部のトレーシング関連プログラム型には `CAP_PERFMON` が要求される [5]．この実装では，特権を要する eBPF 操作は，その実行主体が，ホストで最初に作成される初期ユーザ名前空間において必要なケーパビリティを有している場合に許可される．このため，コンテナプロセスにこれらの操作を直接行わせるには，初期ユーザ名前空間における必要なケーパビリティを付与する必要があり，必要な eBPF 操作の範囲を越えて，ホストに影響し得る操作まで可能にするおそれがある．

この問題を解決するために，Linux 6.9 では BPT Token が導入された．BPF Token は，特権プロセスが信頼する非特権プロセスに対し，許可する eBPF 操作の範囲を限定して権限を委譲するための機構である [2]．権限委譲元は，BPF ファイルシステムのマウントオプションにより，実行可能な `bpf()` システムコールのコマンド，作成可能な BPF マップ型，ロード可能な eBPF プログラム型，およびプログラムのロード時に指定可能なアタッチ型を設定できる．Token は，この BPF ファイルシステムインスタンスから作成され，同インスタンスを所有するユーザ名前空間に関連付けられる．Token を指定した `bpf()` システムコールでは，必要なケーパビリティが Token に関連付けられたユーザ名前空間に対して検査される．これにより，初期ユーザ名前空間における `CAP_BPF` などのケーパビリティをコンテナプロセスに付与せず，必要な eBPF 操作に関する権限を委譲できる．コンテナ基盤における利用例として，LXD では，BPF Token による権限委譲をコンテナインスタンスごとに設定できる [6]．

一方，BPF Token では，BPF マップや eBPF プログラムなどの BPF オブジェクトの個数や作成・ロード頻度の上限を設定できない．また，個々のヘルパーを許可項目として指定できないため，許可されたプログラム型の中で，用途に不要なヘルパーも利用可能となる場合がある．さらに，アタッチに関して指定できるのはプログラムのロード時のアタッチ型までであり，具体的なアタッチ対象とその実効範囲を制限できない．本研究は，BPF オブジェクト作

© 2026 Information Processing Society of Japan

成による資源消費，ヘルパー利用範囲，およびアタッチ対象とその実効範囲について，BPF Token による正規の権限委譲後にも残るコンテナや Pod ごとのポリシー表現上の限界を対象とする．

2.2 BPF LSM

LSM は，カーネル内のセキュリティフックを通じてアクセス制御を実装するための枠組みである．BPF LSM では，eBPF プログラムをこれらのフックにアタッチすることで，カーネルを変更することなくアクセス制御ポリシーを実装できる [7]．BPF LSM を用いる構成では，BPF マップをポリシーの格納領域とし，ユーザ空間の信頼されたポリシー設定主体がその内容を更新できる．また，eBPF 操作に対応する LSM フックにアタッチされた BPF LSM プログラムでは，その操作を要求したプロセスが属する cgroup の ID などを取得し，適用するポリシーの選択に利用できる．

本研究では，この構成を用いて，BPF Token を保持するコンテナプロセスが直接行う eBPF 操作に対し，cgroup ID ごとのポリシーを適用する方式を設計・実装する．

2.3 eBPF の安全な利用とコンテナ隔離に関する既存研究

He らは，コンテナプロセスがロードしたトレーシング用途の eBPF プログラムが，コンテナ外のプロセスに関するイベントを契機としても実行され，同プログラムからヘルパーを呼び出すことで，対象プロセスの情報取得や状態変更が可能になることを示した [1]．また，公開コンテナイメージの実行設定やクラウドサービスを調査し，eBPF を実行できる過剰な権限がコンテナプロセスに付与される事例を明らかにした．同研究では，プロセスごとに eBPF 機能の利用可否と，eBPF からの干渉を受けるか否かを表すビットを追加する CapBits を提案している．CapBits は細粒度な制御を実現する一方，カーネルのデータ構造と権限検査処理の変更を必要とする．

KRAKENGUARD は，非特権プロセスから受け取った eBPF バイトコードについて，プログラムの入力を記号として扱って実行経路を解析するシンボリック実行によりポリシー適合性を確認したうえで，特権デーモンが代理でロードする [8]．利用可能なヘルパー，BPF マップごとの読取り・書込みの可否，プログラム入力に対応するメモリ領域のうち読取り・書込みを許可する範囲，および許可する戻り値を，プログラムごとに指定できる．さらに，プログラム内の分岐によって生じる到達可能な各実行経路について，メモリや BPF マップへのアクセスを解析し，複数の eBPF プログラム間の干渉も検査できる．

BBArmor は，BPF LSM プログラムを `bpf()` システムコールの処理で呼び出される LSM フックにアタッチし，名前空間ごとに設定した規則に基づいて，プログラム型，ヘルパー呼出し，およびアタッチ種別などを検査する [9]．さらに，BPF LSM プログラムを `perf_event_open()` システムコールの処理で呼び出される LSM フックにもアタッチし，指定されたトレースポイント ID，または kprobe の設置先となるカーネル関数と関数内オフセットに対応する perf event の作成要求を検査する．また，BBArmor は，BPF Token の発行後には，同じ Token 保持者による個々の eBPF 操作を動的に選別できないという問題を指摘している．

BPFContain は，ファイル，ネットワーク，ケーパビリティなどのシステム資源に対するアクセス規則をコンテナごとに記述する YAML 形式のポリシーと，BPF LSM を含む eBPF プログラムを組み合わせ，ポリシーをカーネル空間で適用する [10]．特権デーモンが eBPF プログラムとポリシーをロードし，ラッパープログラムが対象アプリケーションをポリシーに対応付けて起動する．さらに，対象プロセスとその子孫を独自のコンテナ ID に対応付けて追跡することで，ユーザ空間におけるコンテナ管理とカーネル空間におけるポリシー適用を結び付ける．ただし，同方式はコンテナプロセスによるシステム資源へのアクセスを制御し，`bpf()` の利用自体を暗黙のポリシーで拒否するため，BPF Token による権限委譲後の eBPF 操作を選別するものではない．

KRAKENGUARD の代理ロード・ロード前検査と BBArmor の BPF LSM による検査は，それぞれ本研究のユーザ空間ブローカー方式と BPF LSM 方式の一部と重複する．一方，以上の既存研究は，BPF Token を実際に用いた権限委譲後の制御限界について，資源消費，ヘルパー利用範囲，およびアタッチ対象とその実効範囲の 3 つの観点から，cgroup ID ごとのポリシー適用可否を検証するものではない．本研究は，これらの制御限界を補完するユーザ空間ブローカー方式と BPF LSM 方式を実装し，同一の実験環境で適用可能なポリシー，保護範囲，および処理コストを比較する点に特徴がある．

3. 問題設定と想定環境

3.1 制御対象と研究課題

本研究では，システム監視，ネットワーク制御，セキュリティ監視などの役割を担い，BPF Token によって eBPF 操作に関する権限を正当に委譲されたエージェントを想定する．エージェントは，これらの役割の遂行において，eBPF 操作だけでなく，ソケット処理，性能計測，プロセス監視，他プロセスとの通信などを担当し，BPF Token による権限とは独立してソケットまたは perf event を取得または指定する権限を持つ場合がある．このエージェントが侵害されると，個別には正当に付与された権限の組合せによって，本来の役割を超えた操作が行われ得る．そこで，本研究では，Token が許可する操作に対して cgroup ID ごとの追加

© 2026 Information Processing Society of Japan

のポリシーを適用できるかを，資源消費，ヘルパー利用範囲，およびアタッチ対象とその実効範囲の 3 つの制御対象について検証する．

資源消費については，BPF オブジェクトの個数や容量，作成およびロードの頻度を制限できるかを検証する．ヘルパー利用範囲については，許可されたプログラム型の中で，用途に不要なヘルパーの利用を制限できるかを検証する．アタッチ対象とその実効範囲については，正規に取得または指定可能な対象集合が，役割上許可されたアタッチ対象集合より広い場合に，両権限の組合せによる許可対象外へのアタッチを制限できるか，また，アタッチの結果としてプログラムが観測または作用できる範囲を制限できるかを検証する．その代表例として，ソケットへのアタッチおよびトレースポイントを用いたトレーシングを扱う．

これらについて，まず，BPF Token 単独では cgroup ID ごとのポリシーを適用できないことを実験により明らかにする．次に，ユーザ空間ブローカー方式と BPF LSM 方式の適用可能なポリシー，保護範囲，および処理コストを比較する．

3.2 想定環境と脅威モデル

本研究が想定するエージェントは，コンテナプロセスとして動作し，初期ユーザ名前空間とは異なるユーザ名前空間，初期マウント名前空間とは異なるマウント名前空間，およびコンテナや Pod に対応する cgroup に属する．信頼された権限委譲元プロセスは，BPF Token 機構を用いて，エージェントの役割に必要な eBPF 操作に関する権限を正当に委譲する．また，信頼されたポリシー設定主体は，エージェントの役割に応じて，資源上限，利用可能なヘルパー，および許可するアタッチ対象などのポリシーを設定するものとする．

この環境における問題を検証するには，Token に関連付けるユーザ名前空間，その作成元となる BPF ファイルシステムを配置するマウント名前空間，およびポリシーの管理キーとなる cgroup ID を再現する必要がある．そこで，本研究では，初期ユーザ名前空間とは異なるユーザ名前空間および初期マウント名前空間とは異なるマウント名前空間で実行され，専用の cgroup に属する実験用プロセスを用いる．本稿では，このプロセスを疑似コンテナプロセスと呼ぶ．本研究では，疑似コンテナプロセスが属する cgroup の ID をポリシーの管理キーとして用い，cgroup ID ごとのポリシー適用可否を検証する．ただし，実際のコンテナ基盤との連携や，各種の隔離・資源制御機構およびコンテナイメージを基にしたファイルシステムからなるコンテナ実行環境全体の再現は行わない．**図 1** に，疑似コンテナプロセスの生成過程と信頼境界を示す．

攻撃者は，正当に権限委譲されたエージェントを模擬する疑似コンテナプロセスを制御し，Token の許可範囲内で

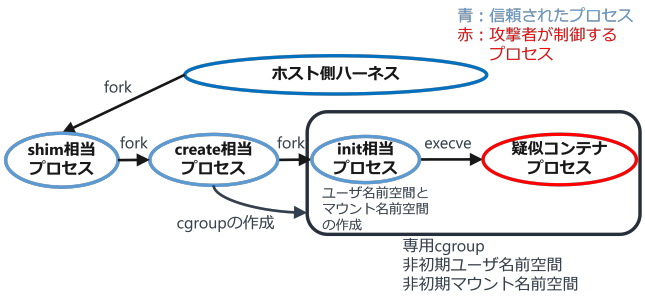


図 1 実験環境におけるプロセス生成の流れと信頼境界
Fig. 1 Process creation flow and trust boundary in the experimental environment

eBPF 操作を直接試行するか，ブローカーへ任意の操作を要求できるものとする．アタッチ対象とその実効範囲の検証では，eBPF とは別の正当な用途で利用可能だが，アタッチ先としては許可されていないソケットまたは perf event へのアタッチを試行する．ただし，攻撃者は，任意のホスト資源のファイル記述子を取得できず，対象の取得に必要な独立した権限検査を回避できないほか，初期ユーザ名前空間の root 権限および eBPF 操作に必要なケーパビリティを有さず，カーネル脆弱性を悪用できないものとする．また，権限委譲元プロセス，後述するブローカー，および BPF LSM 方式のポリシー設定主体は信頼できるものとする．

4. BPF Token の制御限界を補完する方式

4.1 全体構成

本研究では，BPF Token による権限委譲後の制御限界を補完する 2 つの方式を，BPF Token 単独方式を基準として比較する．BPF Token 単独方式（A1）では，疑似コンテナプロセスが BPF Token を保持し，Token が許可する eBPF 操作を `bpf()` などの既存のインタフェースを通じて直接行う．ユーザ空間ブローカー方式（A2）では，疑似コンテナプロセスに Token を渡さず，信頼されたブローカーが同プロセスからの操作要求を検査して eBPF 操作を代行する．BPF LSM 方式（A3）では，疑似コンテナプロセスが Token を保持して eBPF 操作を直接行い，その操作に対して，cgroup ID ごとのポリシーを BPF LSM が適用する．**図 2** に，各方式の BPF Token 保持主体と eBPF 操作経路を示す．

実運用では，コンテナ基盤と連携する信頼されたポリシー設定主体が，コンテナや Pod に対応する cgroup を特定し，その cgroup ID をポリシーの管理キーとして用いることを想定する．本研究では，実際のコンテナ基盤との連携は行わず，疑似コンテナプロセスを配置した cgroup の ID を，A2 および A3 のポリシーの管理キーとして用いる．実験では，信頼されたセットアップ主体が保護対象操作の開始前に専用の cgroup を作成して疑似コンテナプロセス

© 2026 Information Processing Society of Japan

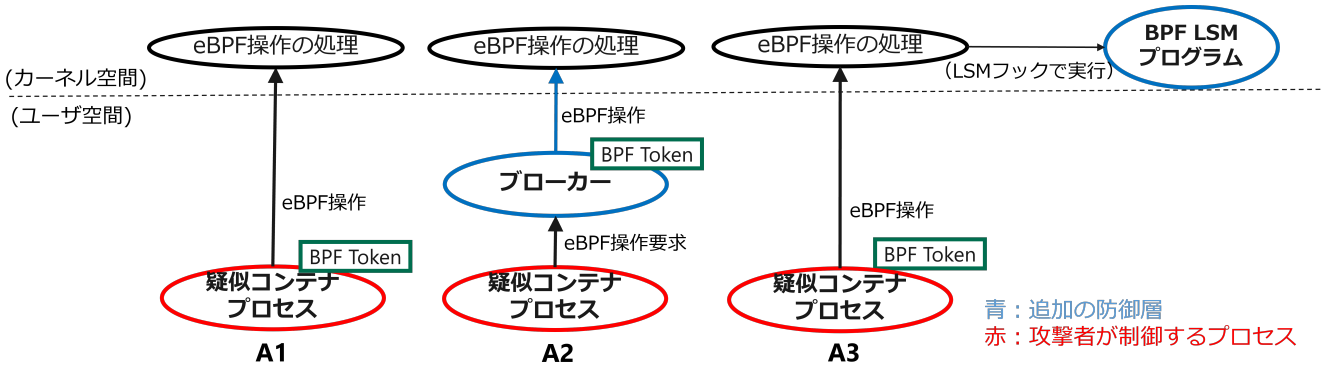


図 2　各方式の BPF Token 保持主体と eBPF 操作経路
Fig. 2　BPF Token holders and eBPF operation paths in each mechanism

を配置し，その cgroup ID を確認する．A2 は確立した要求経路をこの ID に結び付け，A3 は LSM フックから操作主体の cgroup ID を取得するため，いずれも要求元の自己申告値を管理キーとして用いない．

BPF Token に設定された BPF コマンド，BPF マップ型，eBPF プログラム型，およびアタッチ型の制限は，A1，A2，A3 のいずれにも適用される．A2 と A3 の相違は，追加のポリシーを適用する位置と，判定に用いる情報にある．A2 は，ユーザ空間のブローカーにおいて，操作要求の内容と，要求元の疑似コンテナプロセスの cgroup ID に対応するポリシーおよび状態に基づいて判定する．A3 は，LSM フックで実行される BPF LSM プログラムにおいて，eBPF 操作を行う疑似コンテナプロセスの cgroup ID に対応するポリシーおよび状態と，フックから観測可能な情報に基づいて判定する．

4.2　ユーザ空間ブローカー方式

A2 では，信頼されたブローカーが，BPF Token のファイル記述子と，自身が作成した BPF オブジェクトのファイル記述子を保持する．疑似コンテナプロセスは，マップ作成，プログラムロード，ソケットへのアタッチ，およびトレーシングの設定などをブローカーに要求する．ブローカーは要求がポリシーに適合する場合のみ，自身が保持する Token を用いた BPF オブジェクトの作成や，その後のアタッチなどの eBPF 操作を代行する．

マップ作成要求に対しては，管理する未解放マップ数および作成頻度をポリシーの上限と比較するとともに，要求されたマップの論理容量が同ポリシーの上限以内であるかを検査する．いずれかがポリシーに適合しない場合は拒否する．プログラムロード要求に対しては，管理する未解放プログラム数およびロード頻度をポリシーの上限と比較する．また，要求されたプログラム型に対応するヘルパープロファイルを参照し，ロード対象プログラムの命令列に含まれるヘルパー呼出しを同プロファイルと照合する．いずれかがポリシーに適合しない場合は拒否する．ソケットへのアタッチ要求に対しては，要求されたソケットのファイル記述子について，アドレスファミリ，ソケットタイプ，プロトコル，およびネットワーク名前空間を検査し，対応するポリシーに登録されたネットワーク名前空間に属する IPv4 UDP ソケットのみを許可する．トレーシングの設定要求に対しては，要求されたトレースポイント，トレース対象プロセス，およびトレース対象 CPU を検査し，対応するポリシーに登録された値と一致する場合だけ許可する．

A2 については，通信方式の異なる A2-1 と A2-2 を実装した．A2-1 は操作ごとの要求と応答に Unix ドメインソケットを用い，A2-2 は共有メモリと `futex` による通知を用いる．A2-2 でも，初期設定およびファイル記述子の受渡しには Unix ドメインソケットを用いる．ソケットへのアタッチでは，A2-1 はファイル記述子を含む 1 回の要求と応答を Unix ドメインソケットを介して行い，A2-2 は Unix ドメインソケットによる 1 方向のファイル記述子の受渡しと，共有メモリ上の要求および完了通知を組み合わせる．A2-1 と A2-2 は同じ Token 保持主体およびポリシーを用い，通信方式のみが異なる．

4.3　BPF LSM 方式

A3 では，信頼されたポリシー設定主体が，疑似コンテナプロセスの cgroup ID をキーとして，適用するポリシーを BPF マップへ登録する．本方式では，制御対象とする eBPF 操作の処理経路を調査し，ポリシーの適用に利用可能な LSM フックを選定して，BPF LSM プログラムを各フックにアタッチする．同プログラムには，eBPF 操作を行う疑似コンテナプロセスの cgroup ID に対応するポリシーおよび状態を参照して，操作を許可または拒否する処理を実装する．疑似コンテナプロセスは A1 と同様に Token を保持し，`bpf()` などの既存の操作インタフェースを通じて eBPF 操作を直接行う．

マップ作成に対しては，管理する未解放マップ数および作成頻度をポリシーの上限と比較するとともに，フックから観測されるマップの論理容量が同ポリシーの上限以内で

© 2026 Information Processing Society of Japan

あるかを検査する．いずれかがポリシーに適合しない場合は拒否する．プログラムロードに対しては，管理する未解放プログラム数およびロード頻度をポリシーの上限と比較するとともに，フックから観測されるプログラム型に対応するヘルパープロファイルを参照し，命令列に含まれるヘルパー呼出しを同プロファイルと照合する．いずれかがポリシーに適合しない場合は拒否する．また，BPF マップまたは eBPF プログラムの解放時には，対応する未解放数を更新する．ソケットへのアタッチに対しては，ソケットのアドレスファミリ，ソケットタイプ，プロトコル，およびネットワーク名前空間を検査し，対応するポリシーに登録されたネットワーク名前空間に属する IPv4 UDP ソケットのみを許可する．トレーシングの設定に対しては，指定されたトレースポイント ID およびトレース対象 CPU を検査し，ポリシーに登録された値と一致する場合だけ許可する．

5. 評価

本節では，BPF Token 単独方式（A1）の制御限界と，ユーザ空間ブローカー方式（A2）および BPF LSM 方式（A3）による補完範囲を検証し，各方式の処理コストを比較した結果を示す．

5.1 実験環境と評価方法

実験には，Ubuntu 26.04 LTS，Linux 7.0.0-27-generic，bpftool 7.7.0，libbpf-dev 1.6.3 を導入した単一の Linux 仮想マシンを用いた．各試行では疑似コンテナプロセスごとに専用の cgroup を作成し，A2 および A3 ではその cgroup ID をポリシーの管理キーとして用いた．ただし，実際のコンテナ基盤におけるコンテナまたは Pod と cgroup ID との対応付けは評価しない．本実験において，非特権プロセスによるユーザ名前空間内でのケーパビリティ利用と perf event 作成は必須であるため，AppArmor および perf event に関するカーネル設定を一時的に変更したが，これらは BPF Token の許可集合と A2 および A3 のポリシー内容を変更するものではない（付録 A.2）．

制御限界と防御効果の検証では，まず，BPF Token の許可集合に含まれる操作と含まれない操作を実行し，Token による制限が有効であることを確認した．そして，3 つの制御対象について，ポリシーに適合する操作と適合しない操作を実行し，各方式が許可または拒否できる範囲を確認した．これらは防御効果を確認する機能試験であり，処理コストの集計には含めない．

処理コストの比較では，疑似コンテナプロセスが対象操作を開始してから，その結果を受け取るまでを計測区間とした．具体的には，計測区間には，各方式の eBPF 操作に加え，A2 の要求・応答と要求検査，および A3 の LSM フックにおけるポリシー検索と判定を含めた．一方，操作前の準備と各試行後の終了処理は含めない（付録 A.2）．各方式と操作の組合せを 100 回独立に試行し，実行順序を切り替えて処理時間の中央値と 95 パーセンタイルを求めた．同一のベンチマークプログラムを用い，疑似コンテナプロセスのライフサイクルと各操作の入力条件を方式間で統一した．

5.2 適用可能なポリシーと保護範囲

BPF Token の許可集合について検証した範囲では，許可された BPF コマンド，マップ型，プログラム型，およびプログラムロード時のアタッチ型を指定した操作は成功し，許可されていないものを指定した操作は拒否された．したがって，以下の結果は Token の許可集合が無効であることを示すものではなく，許可された操作に対して残るポリシー表現上の限界を示す．**表 1** に，3 つの制御対象に対する各方式の結果を示す．

資源消費については，A2 および A3 のいずれも，未解放のマップ数およびプログラム数，マップの論理容量，ならびにマップ作成およびプログラムロード頻度に上限を適用できた．具体的には，上限内のマップ作成およびプログラムロードを許可し，未解放数，論理容量，または頻度の上限を超える操作を拒否した．

ヘルパー利用範囲について，A2 では，識別情報取得ヘルパーを用いるプログラムロードを許可し，ユーザメモリ読取りヘルパーを含む要求を `BPF_PROG_LOAD` の前に拒否した．ただし，マップ参照ヘルパーについては要求検査における許可だけを確認し，実際のプログラムロードは行っていない．A3 では，識別情報取得ヘルパーまたはマップ参照ヘルパーを用いるプログラムロードを許可し，ユーザメモリ読取りヘルパーを含むプログラムを `BPF_PROG_LOAD` 中に拒否した．

アタッチ対象とその実効範囲については，A2 および A3 のいずれも，許可したネットワーク名前空間の IPv4 UDP ソケットへのアタッチを許可し，Unix ストリームソケットおよび異なるネットワーク名前空間の IPv4 UDP ソケットへのアタッチを拒否した．ただし，両方式とも，ソケットの作成主体やファイル記述子の受渡し履歴は検査できない．トレーシングでは，A2 はトレースポイント，トレース対象プロセス，およびトレース対象 CPU を要求から検査し，不一致の要求を `perf_event_open` の前に拒否した．A3 は，perf event の作成時にトレースポイント ID およびトレース対象 CPU に関するポリシーを適用できたが，ポリシー適用に用いた LSM フックからトレース対象プロセスを識別できない．さらに，既存の perf event のファイル記述子への後続のプログラムアタッチは再検査しないため，A2 と同一のポリシーを適用できるわけではない．

以上から，検証した範囲では，A2 および A3 のいずれも，BPF Token 単独方式では表現できない 3 つの制御対

© 2026 Information Processing Society of Japan

表 1 3 つの制御対象に対する各方式の制御範囲
Table 1 Policy coverage of each mechanism across the three control dimensions

制御対象＼方式	A1	A2	A3
資源消費	なし	未解放数・論理容量・作成／ロード頻度	未解放数・論理容量・作成／ロード頻度
ヘルパー利用範囲	なし	プログラム型ごとのヘルパー	プログラム型ごとのヘルパー
アタッチ対象とその実効範囲	なし	ソケット属性・ネットワーク名前空間，トレースポイント ID・トレース対象プロセス・トレース対象 CPU	ソケット属性・ネットワーク名前空間，トレースポイント ID・トレース対象 CPU

表 2 各方式の処理時間（中央値／ 95 パーセンタイル，単位：μs（トレースポイントへのアタッチを除く））
Table 2 Processing times of each mechanism (median/95th percentile, in μs excepting tracepoint attachment)

操作＼方式	A1	A2-1	A2-2	A3
マップ作成	24 / 34	1,143 / 1,451	532 / 815	42 / 52
プログラムロード（ヘルパーなし）	79 / 101	1,165 / 1,542	579 / 847	116 / 142
プログラムロード（識別情報取得ヘルパーあり）	110 / 164	1,223 / 1,725	604 / 835	156 / 197
ソケットへのアタッチ	3 / 4	842 / 1,159	674 / 889	4 / 5
トレースポイントへのアタッチ	2 ms / 3 ms	3 ms / 4 ms	12 ms / 19 ms	9 ms / 17 ms

象に追加のポリシーを適用できた．適用可能なポリシーについて，A2 は操作要求に含まれる情報を検査するため，トレース対象プロセスなど LSM フックからは得られない情報を判定に利用できる．一方，保護範囲について，A2 はブローカーを経由する操作を対象とするのに対し，A3 は対象とする LSM フックを通る eBPF 操作にカーネル空間でポリシーを適用するため，特定のユーザ空間インタフェースを経由することを前提としない．ただし，A3 で表現できるポリシーと保護できる操作経路は，LSM フックの配置と同フックから観測できる情報に制約される．

5.3 処理コスト

表 2 に，各方式でポリシーが許可する操作の処理時間を示す．各方式と各操作の組合せからなる 20 条件について，それぞれ 100 回，合計 2,000 回実行し，全試行で対象操作が成功した．

マップ作成，プログラムロード，およびソケットへのアタッチでは，A2-1 の中央値は 842–1,223 μs であった．A2-2 は，A2-1 と同じ要求検査を行いながら，通信経路を共有メモリと `futex` に変更することで，これらの中央値を 532–674 μs に削減した．この結果から，A2-1 の処理コストには，Unix ドメインソケットによる要求・応答処理が大きく寄与したと考えられる．A2-2 の処理時間は A1 および A3 より依然として大きいが，A2 では保護範囲を変えずに通信方式の変更によって処理コストを低減できることを確認した．

A3 と A1 の中央値の差は，マップ作成，プログラムロード，およびソケットへのアタッチで 1–46 μs であった．この結果から，A3 は，疑似コンテナプロセスによる直接的な操作経路を維持しながら，これらの操作では A1 に近い処理時間でポリシーを適用できることを確認した．

トレースポイントへのアタッチでは，A1，A2-1，A2-2，および A3 の中央値は，それぞれ 2 ms，3 ms，12 ms，および 9 ms であった．この計測には，各方式に固有の要求処理およびポリシー適用と，`perf_event_open` による perf event の作成，eBPF プログラムのアタッチ，ならびに perf event の有効化を含め，perf event の解放は含めていない．したがって，本結果は，ポリシーの検査と判定だけでなく，各方式でアタッチが完了するまでの一連の処理時間を示す．

これらは，eBPF プログラムの実行時ではなく，マップ作成，プログラムロード，およびアタッチといった設定時の操作に対する処理時間である．したがって，A2 の処理コストによる影響はこれらの操作頻度に依存し，低頻度の初期設定と，頻繁に構成を変更する運用とでは評価が異なる．

以上の防御効果と処理コストの結果を踏まえると，A2 と A3 は一方が常に優位な方式ではなく，ポリシーの判定に必要な情報と許容できる処理コストに応じて選択する必要がある．また，両方式は排他的ではなく，フックから得られる情報で表現できるポリシーには A3 を用い，それ以外の文脈を要する操作には A2 を用いる構成も考えられる．

5.4 制約

A2 は，疑似コンテナプロセスにブローカー固有の要求 API を用いさせ，別の Token の取得やブローカーを介さな

© 2026 Information Processing Society of Japan

い eBPF 操作を防ぐ構成を前提とする．

A3 の資源計数は LSM フックの実行順序に依存し，計上後に別の LSM モジュールが操作を拒否しても計数状態は巻き戻されない．また，オブジェクトの対応付けに用いるカーネル内部情報のバージョン間安定性は評価していない．

cgroup ID をキーとするポリシー分離と同一 cgroup 内の並行操作を補足的に確認したが（付録 A.3），実際のコンテナ基盤との対応付け，ポリシーのライフサイクル，cgroup 間の移動・ID 再利用，および障害後の状態回復は評価していない．

資源消費は `ARRAY` マップと `TRACEPOINT` プログラムに限定し，実際のカーネルメモリ使用量や攻撃的な負荷下の可用性を評価していないため，実運用環境全体の DoS 耐性を実証するものではない．また，ヘルパーには実験用の許可リストを用い，アタッチ先をソケットとトレースポイントに限定したため，他の設定への一般化には追加検証を要する．

処理コストは，単一仮想マシンおよび単一のカーネルバージョン上で同時実行負荷を与えずに測定した設定時操作の相対値であり，実運用環境や eBPF プログラム実行時の処理コストを示すものではない．

6. おわりに

本研究では，BPF Token によって eBPF 操作に関する権限が正当に委譲された後に残る制御限界を，資源消費，ヘルパー利用範囲，およびアタッチ対象とその実効範囲の 3 つの観点から検証した．その結果，BPF Token 単独方式では，これらに対する cgroup ID ごとのポリシーを表現できないことを明らかにした．この制御限界を補完するため，ユーザ空間ブローカー方式と BPF LSM 方式を設計・実装し，いずれの方式でも 3 つの制御対象に追加のポリシーを適用できることを確認した．

ユーザ空間ブローカー方式は，操作要求に含まれる情報を利用し，LSM フックからは得られない情報も判定に用いることができる一方，保護対象がブローカーを経由する操作に限られ，要求処理によるコストを伴う．BPF LSM 方式は，適用可能なポリシーと保護できる操作経路が LSM フックから観測できる情報に制約されるが，疑似コンテナプロセスによる直接的な eBPF 操作に，BPF Token 単独方式に近い処理時間でポリシーを適用できた．以上から，両方式は一方が常に優位なものではなく，ポリシーの判定に必要な情報，保護する操作経路，および許容できる処理コストに応じて選択または組み合わせる必要がある．今後は，実際のコンテナ基盤におけるコンテナまたは Pod と cgroup ID との対応付けを含むポリシー管理，および対象とする eBPF 操作の拡張を検討する．

謝辞　本研究は JSPS 科研費 23K28086，24K14959 の助成を受けたものである．

参考文献

[1] Yi He, Roland Guo, Yunlong Xing, Xijia Che, Kun Sun, Zhuotao Liu, Ke Xu, and Qi Li, “Cross Container Attacks: The Bewildered eBPF on Clouds,” In Proceedings of the 32nd USENIX Security Symposium (USENIX Security ’23), pp.5971–5988, August 2023.

[2] Andrii Nakryiko, “bpf: Introduce BPF token object,” Linux kernel commit 35f96de04127, January 2024. https://github.com/torvalds/linux/commit/35f96de04127d332a5c5e8a155d31f452f88c76d

[3] The Linux Kernel Documentation, “BPF Documentation.” https://docs.kernel.org/bpf/ (accessed July 27, 2026).

[4] Linux man-pages project, “capabilities(7) — overview of Linux capabilities.” https://man7.org/linux/man-pages/man7/capabilities.7.html (accessed July 31, 2026).

[5] Alexei Starovoitov, “bpf, capability: Introduce CAP_BPF,” Linux kernel commit a17b53c4a4b5, May 2020. https://github.com/torvalds/linux/commit/a17b53c4a4b55ec322c132b6670743612229ee9c

[6] Canonical, “Privilege delegation using BPF Token,” LXD Documentation. https://canonical.com/lxd/docs/latest/explanation/bpf/ (accessed July 31, 2026).

[7] The Linux Kernel Documentation, “LSM BPF Programs.” https://docs.kernel.org/bpf/prog_lsm.html (accessed July 27, 2026).

[8] Jainil Patel, Lucas Graeff Buhl-Nielsen, Adrien Ghosn, and Marios Kogias, “KRAKENGUARD: Towards Fine-Grained eBPF Isolation,” In Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI ’26), pp.2685–2704, May 2026.

[9] Fabio Piras, Giuseppe Lettieri, and Gregorio Procissi, “BBArmor: a Dynamic BPF-to-BPF LSM-Based Enforcement Tool,” In Proceedings of the 21st International Conference on Network and Service Management (CNSM 2025), 2025. https://dl.ifip.org/db/conf/cnsm/cnsm2025/1571197273.pdf

[10] William Findlay, David Barrera, and Anil Somayaji, “BPFContain: Fixing the Soft Underbelly of Container Security,” arXiv preprint arXiv:2102.06972, February 2021.

© 2026 Information Processing Society of Japan

7. 研究倫理に関する検討

本研究において実施した実験は，著者らが管理する単一の仮想マシンでのみ実施し，第三者のシステム，サービス，データを対象としていない．そのため，本研究の実施そのものによる第三者への危害は極めて限定的である．

本論文の公開は，BPF Token の制御限界の悪用が成立する条件の把握・再現を容易にする可能性がある．一方で，本研究は BPF Token 自体の不正な発行，アタッチ対象の利用に別途必要な権限検査の回避，またはカーネル脆弱性の悪用を可能にするものではない．したがって，本論文の公開は，既に成立条件を満たす限定的な環境における問題の発見・再現能力の増加であり，権限を持たない第三者の攻撃能力を直ちに高めるものではない．また，本論文は制御限界だけでなく，2 つの補完方式とその適用範囲および制約も示しており，その公開には，利用者・開発者が追加制御や文書化，設計改善を検討できるという防御上の利益がある．

本論文が扱う技術的詳細の公開範囲については，具体的な条件や実装方法を公開することで，悪用の再現を容易にするおそれがある．一方で，本論文が主張する問題の理解，再現性の確保には技術的詳細の公開が必要である．これらの悪用リスクと公益のバランスを考慮し，製品固有の侵害手順，攻撃の自動化，攻撃的な負荷設定といった内容は含めず，研究上必要な範囲に限定して技術的詳細を公開した．

本研究の実験で用いた検証用実装の公開は，再現性の確保や防御方式の検証に有用である一方，論文のみを公開する場合よりも，悪用の再現を容易にする可能性がある．したがって，論文の公開とは別に，公開可否，公開範囲，公開時期を検討することが必要である．現段階では，検証用実装は公開しないと判断した．公開すると判断した場合には，公開する機能や設定例の範囲，公開時期，および隔離環境で利用することなどの安全な利用条件を改めて検討する．

本研究は特定の製品に固有の欠陥や BPF Token の許可集合を突破する挙動を示すものではないため，特定のベンダへの脆弱性届出は必要ないと判断した．関連する Linux カーネル開発者への事前共有には，研究結果の技術的妥当性の確認，文書化や注意喚起を検討する機会の提供などの意義がある．しかし，前述した公開による悪用能力への影響と公開範囲に関する検討を踏まえ，本論文の公開前に緊急の修正や注意喚起を検討する期間を設ける必要性は低いと判断し，事前共有は行わなかった．

以上の検討および研究倫理相談窓口からの助言を踏まえ，論文公開による悪用能力の実質的な増加は限定的であり，防御上の公益がこれを上回ることから，論文としての公開に支障はないと判断した．

© 2026 Information Processing Society of Japan

付　　録

A.1　補完方式の実装詳細

本付録では，第 4 節で示した A2 および A3 について，ポリシーの管理キーを確立してから操作を許可または拒否するまでの実装を補足する．

A.1.1　実験用ランタイム構成

本研究の実装では，containerd と runc を用いる構成を参考に，高水準コンテナランタイムの役割を模したホスト側ハーネス，長寿命の shim 相当プロセス，一時的な低水準コンテナランタイムの create 相当プロセス，init 相当プロセス，および疑似コンテナプロセスからなる構成を用いる．ホスト側ハーネスは shim 相当プロセスを生成し，shim 相当プロセスが create 相当プロセスを生成する．

create 相当プロセスは, 疑似コンテナプロセス用の cgroup を作成し，初期化処理の開始を待機させた init 相当プロセスを生成して同 cgroup へ配置する．create 相当プロセスは cgroup ID を shim 相当プロセスへ通知し，init 相当プロセスを再開してから終了する．init 相当プロセスは，初期ユーザ名前空間とは異なるユーザ名前空間および専用のマウント名前空間で bpffs を準備した後，同一の実験バイナリを，疑似コンテナプロセスとして動作する内部モードで `execv` する．長寿命の shim 相当プロセスは，create 相当プロセスの終了後も init 相当プロセスを追跡し，その終了後に cgroup を削除する．

信頼されたポリシー設定主体を含むホスト側ハーネスは, 通知された cgroup ID と init 相当プロセスの所属先が一致することを確認する．その後，A2 では要求経路を同 ID に結び付け，A3 では同 ID をキーとするポリシーを BPF マップへ登録し，これらの処理が完了するまで保護対象の操作を開始させない．この構成はコンテナランタイムのプロセス間の役割とライフサイクルを模したものであり，実際の containerd または runc との連携や OCI コンテナの生成は行わない．

A.1.2　ユーザ空間ブローカー方式

A2 のブローカーは, 確立した要求経路に対応する cgroup ID をセッション状態として保持する．疑似コンテナプロセスから受け取る要求には，マップまたはプログラムの作成条件，プログラムの命令列，ソケットのファイル記述子，あるいはトレーシングの設定値など，対象操作の判定に必要な情報を含める．ブローカーは，要求元が申告する識別子ではなく，要求経路に結び付けた cgroup ID に対応するポリシーおよび状態を参照する．

A2-1 では，操作ごとに Unix ドメインソケットを介して要求と応答を交換し，ファイル記述子は `SCM_RIGHTS` を用いて受け渡す．A2-2 では，要求と完了通知に共有メモリ上の有界なリングバッファと `futex` を用いる．共有メモリではファイル記述子を受け渡せないため，ソケットへのアタッチでは，Unix ドメインソケットによる一方向のファイル記述子転送と，共有メモリ上の要求を識別子によって対応付ける．

資源消費に関して，ブローカーは，作成に成功した BPF マップおよび eBPF プログラムのファイル記述子を保持し，cgroup ID ごとに未解放数，マップの論理容量，および固定時間窓内の試行回数を管理する．作成またはロードが成功した場合だけ未解放数と論理容量を加算し，セッションの終了時に保持するファイル記述子を閉じて状態を解放する．一方，頻度の計数は操作の試行時に更新し，作成したオブジェクトが解放されても減算しない．

A.1.3　BPF LSM 方式

A3 では，信頼されたポリシー設定主体が，cgroup ID をキーとするポリシーを事前に BPF マップへ登録する．各 BPF LSM プログラムは，フックが呼び出された操作の主体について cgroup ID を取得し，対応するポリシーが存在しない場合は対象操作を拒否する．**表 A·1** に，制御内容と LSM フック，判定に用いる情報，および管理状態の対応を示す．

BPF マップについては，`bpf_map_create` で観測できる cookie をオブジェクトの対応付けに用い，`bpf_map_free` で実際の解放を観測した時点で未解放数と論理容量を減算する．eBPF プログラムについては，検証処理によって外側の `struct bpf_prog` が再配置される場合があるため，ロード時から解放時まで維持された `struct bpf_prog_aux` へのポインタを対応付けに用いる．これらは対象カーネルの BTF（BPF Type Format）から参照可能な内部情報に依存するため，異なるカーネルで利用する場合には再検証が必要である．

A.1.4　実装コード量

表 A·2 に，A1 に対して追加した補完防御の実装規模を示す．集計対象は，保管防御の実現に不可欠であり，本文に掲載した保護範囲または処理コストの評価で実際に使用されるソースコードに限定した．評価実験スクリプト，集計スクリプト，ビルド定義，ビルド成果物，一連のドキュメントなどは除外した．全行数は空行およびコメント行を含む行数であり，コード行数は空行およびコメントのみからなる行を除いた行数である．

方式ごとのコード行数は，共通部分を含めて A2-1 が 1,408 行，A2-2 が 1,622 行，A3 が 1,047 行である．

© 2026 Information Processing Society of Japan

表 A·1　A3 における制御内容と LSM フックの対応
Table A·1　Mapping between A3 controls and LSM hooks

制御内容	LSM フック	参照情報，ポリシーおよび状態
BPF マップの作成・解放	`bpf_map_create` `bpf_map_free`	Token の有無，マップ型，`value_size`，`max_entries` を参照．cgroup ID ごとの未解放数，論理容量，作成頻度，ならびに cookie と作成元 cgroup ID の対応を管理
eBPF プログラムのロード・解放	`bpf_prog_load` `bpf_prog_free`	Token の有無，プログラム型，命令列を参照．cgroup ID ごとの未解放数およびロード頻度，プログラム型ごとのヘルパープロファイル，ならびに `struct bpf_prog_aux` へのポインタと作成元 cgroup ID の対応を管理
ソケットへのアタッチ	`socket_setsockopt`	アドレスファミリ，ソケットタイプ，プロトコル，ネットワーク名前空間の cookie を参照．cgroup ID ごとに許可するソケット条件およびネットワーク名前空間を管理
perf event の作成	`perf_event_alloc`	perf event の種別，トレースポイント ID，CPU を参照．cgroup ID ごとに許可するトレースポイント ID および CPU を管理

表 A·2　補完方式の実装コード量
Table A·2　Source-code size of the complementary mechanisms

構成要素	主な内容	ファイル数	全行数	コード行数
A2 共通	ポリシー管理，要求検査，ブローカー，eBPF 操作の代行	15	1,373	1,204
A2-1 通信	Unix ドメインソケットおよびファイル記述子転送	2	226	204
A2-2 通信	共有メモリ上の要求・完了リング，futex，およびファイル記述子転送	4	465	418
A3 ユーザ空間制御	BPF LSM プログラムのロード・アタッチとポリシーの登録・削除	4	245	210
A3 資源消費（M1）	上限判定，状態管理，および LSM フックでの適用	4	457	409
A3 ヘルパー利用範囲（M2）	命令列の検査および LSM フックでの適用	4	227	191
A3 アタッチ対象とその実効範囲（M3）	ソケットおよびトレーシングに対する判定と適用	6	287	237
合計（重複なし）		39	3,280	2,873

A.2　実験条件の詳細

防御範囲の検証および処理コストの測定では，A2-1，A2-2，および A3 に**表 A·3** のポリシーを設定し，4 方式に対して**表 A·4** の入力条件で各操作を実行した．

実験時には，Ubuntu の AppArmor による非特権ユーザ名前空間内でのケーパビリティ利用制限を解除するため，`kernel.apparmor_restrict_unprivileged_userns` を一時的に 0 とした．また，非特権プロセスが perf event を作成できるように，`kernel.perf_event_paranoid` を一時的に −1 とした．

各試行では，新たな疑似コンテナプロセスを起動し，名前空間，bpffs，cgroup，BPF Token，通信経路，およびポリシーを準備した後，対象操作を 1 回実行した．方式と操作の各組合せを 100 回ずつ測定し，実行順序は試行ごとに 1 つずつ循環させた．

疑似コンテナプロセスが各方式の処理を開始する直前から結果を受け取るまでを計測区間とした．A1 では，BPF Token を用いた対象操作の直接実行を含めた．A2-1 および A2-2 では，疑似コンテナプロセスとブローカーの間の要求・応答の通信，ブローカーによる要求検査，ブローカーによる対象操作の実行を含めた．なお，疑似コンテナプロセスからブローカーへの要求のうち，ファイル記述子の転送は，A2-1，A2-2 のいずれも Unix ドメインソケットを用いた．A3 では対象操作の実行，cgroup ID の取得，ポリシーおよび状態の検索，ならびに判定を含めた．

なお，ソケットへのアタッチでは，事前に作成したソケットへの eBPF プログラムのアタッチを対象操作としたため，ソケットの作成は計測区間に含めなかった．一方，トレースポイントへのアタッチでは，`perf_event_open` による perf event の作成から，eBPF プログラムのアタッチおよび perf event の有効化までを一連の対象操作とし，4 方式とも計測区間に含めた．

A.3　実装成立性の補足検証

本文の評価とは別に，A3 の状態管理およびポリシー分離が想定どおり機能するかを補足的に検証した．これらの試験は A3 の実装成立性を補足するものであり，本文の処理時間の集計には含めない．

© 2026 Information Processing Society of Japan

表 A·3　評価に用いたポリシー
Table A·3　Policies used in the evaluation

制御対象	設定値
資源消費	BPF マップ 64 個，eBPF プログラム 512 個，マップの論理容量 32,768 byte，1 秒当たりのマップ作成 10 回およびプログラムロード 8 回
ヘルパー利用範囲	`bpf_get_current_pid_tgid`，`bpf_get_current_uid_gid`，`bpf_get_current_cgroup_id`，`bpf_ktime_get_ns`，`bpf_get_smp_processor_id`，`bpf_get_current_comm`，`bpf_map_lookup_elem` を許可し，それ以外を拒否
ソケットへのアタッチ	疑似コンテナプロセス用のネットワーク名前空間に属する `AF_INET`，`SOCK_DGRAM`，`IPPROTO_UDP` のソケット
トレーシング	`syscalls/sys_enter_getpid`，疑似コンテナプロセス自身，および同プロセスの実行可能 CPU 集合から選択した 1 論理 CPU．A3 では対象プロセスを検査しない

表 A·4　処理コスト測定に用いた操作条件
Table A·4　Operation conditions used in the processing-cost measurements

操作	入力条件
マップ作成	キーサイズ 4 byte，値サイズ 8 byte，1 エントリの `ARRAY` マップを作成
プログラムロード（ヘルパーなし）	2 命令からなる `TRACEPOINT` プログラムをロード
プログラムロード（識別情報取得ヘルパーあり）	識別情報取得ヘルパーを含む 11 命令の `TRACEPOINT` プログラムをロード
ソケットへのアタッチ	事前に作成した IPv4 UDP ソケットへ，事前にロードした `SOCKET_FILTER` プログラムを初めてアタッチ
トレースポイントへのアタッチ	`perf_event_open`，`PERF_EVENT_IOC_SET_BPF`，および perf event の有効化までを実行

A.3.1　オブジェクトのライフサイクル

BPF マップについて，ファイル記述子の複製，bpffs へのピン，および複数マップの同時保持後に参照を解放した．最終参照の解放時に `bpf_map_free` が呼び出され，対応する cookie に基づいて未解放数と論理容量を減算できることを確認した．また，ファイル記述子を閉じた後もピンだけでマップが生存している間は計数を維持し，ピンの削除によって最終参照が失われた後に計数を解放した．ただし，A3 はピン先のパスと BPF マップ自体の対応を検査しない．

eBPF プログラムについても，ファイル記述子の複製および bpffs へのピン後に参照を解放し，`bpf_prog_free` において未解放数を減算できることを確認した．さらに，検証処理によって外側の `struct bpf_prog` が再配置された場合にも，`struct bpf_prog_aux` へのポインタを用いてロード時と解放時を対応付けられた．同ポインタはカーネル内部の BTF 可視情報であり，安定したユーザ空間 ABI ではない．

A.3.2　並行操作と cgroup 間分離

同一 cgroup 内の並行操作について，上限を 8 として，32 スレッドからマップ作成またはプログラムロードを同時に実行した．いずれの試験でも 8 操作を許可して 24 操作を拒否し，許可したオブジェクトの解放後には未解放数および論理容量が 0 に復帰した．状態の検査と更新には `bpf_spin_lock` を用いるが，信頼されたポリシー設定主体によるポリシーの更新および削除は直列に行うことを前提とする．

cgroup 間のポリシー分離については，2 つの兄弟 cgroup に疑似コンテナプロセスを配置し，資源上限，プログラム型ごとのヘルパー，およびソケット条件が異なるポリシーを設定した．各 cgroup では自身のポリシーだけが適用され，対応するエントリがない操作は拒否された．一方，cgroup が消滅してもポリシーエントリは自動的には削除されないため，信頼されたポリシー設定主体による削除が必要である．

A.3.3　複数ポリシーと BPF LSM の実行順序

資源消費，ヘルパー，ソケット，およびトレーシングのポリシーを同時に有効化し，各制御対象について想定した許可および拒否が維持されることを確認した．プログラムロードで資源消費とヘルパーのフックが共通する場合には，ヘルパー利用範囲の検査を先に実行することで，許可しないヘルパーを含むプログラムを資源消費の計数前に拒否した．

また，A3 と独立した拒否プログラムの実行順序を入れ替えて同じ操作を実行した．先に返された拒否結果は維持され，後続する BPF LSM プログラムは実行されなかった．A3 が資源利用を計上した後に別のプログラムが拒否した場合には，最終的な操作は拒否されるものの，A3 の計数

© 2026 Information Processing Society of Japan

状態は巻き戻されなかった．したがって，本結果は実製品における他の LSM モジュールとの任意の組合せを保証するものではない．

以上の補足検証から，対象カーネルと実装したポリシーの範囲では，オブジェクトの実際の生存期間に基づく計数，同一 cgroup 内の並行操作に対する上限の維持，および cgroup ID をキーとするポリシーの分離が可能であることを確認した．一方，ポリシー設定主体の障害からの回復，cgroup ID の再利用，および異なるカーネルでの内部情報の安定性は，本検証の対象外である．

© 2026 Information Processing Society of Japan